Chapter 23: Expression Optimization

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Lecture Slides

Chapter Outline

Why Expression Optimization?

Key Idea

Optimize over **variable-size tree structures** (expressions, programs, formulas) instead of fixed-length vectors.

Core Challenge

The search space is **combinatorially large** and **variable-dimensional**; standard gradient methods do not apply.

We need representations that enforce *syntactic correctness* automatically.

Applications:

- Symbolic regression: discover equations from data
- Program synthesis: generate code to solve a task
- Design optimization: gear trains, circuits, robot controllers
- Feature engineering / model architecture search

Grammars — Definitions

Context-Free Grammar (BNF)

A grammar $\mathcal{G}$ consists of:

- **Nonterminals** (types): symbols to be expanded, e.g. $\mathbb{R}$
- **Terminals**: leaf symbols, e.g. x, 2, $+$
- **Production rules**: type $\mapsto$ expansion
- **Start symbol**: the root type

The symbol | on the right-hand side means *or* (alternative rules):

$$\mathbb{A} \mapsto \text{rapidly} \mid \text{efficiently}$$

is shorthand for two rules: $\mathbb{A} \mapsto$ rapidly and $\mathbb{A} \mapsto$ efficiently.

Grammar for Arithmetic Expressions

Four-Function Calculator Grammar

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$
$$\mathbb{R} \mapsto x$$
$$\mathbb{R} \mapsto \ln(\mathbb{R})$$
$$\mathbb{R} \mapsto 2$$

Key properties:

- $\mathbb{R}$ is the only nonterminal
- Terminals: x, 2, $+$, ln
- Every fully-expanded tree is a valid expression

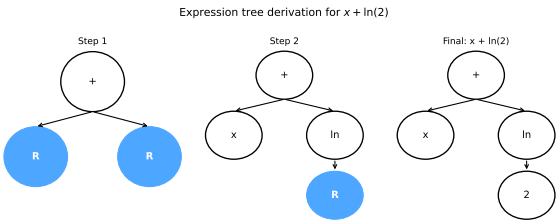

Figure: Derivation of $x + \ln(2)$ using the production rules

Grammar: Formal Structure

Example Context-Free Grammar (BNF)

Production Rule	Interpretation
$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$	R expands to sum of two R's
$\mathbb{R} \mapsto x$	R becomes terminal: x
$\mathbb{R} \mapsto \ln(\mathbb{R})$	R becomes ln applied to R
$\mathbb{R} \mapsto 2$	R becomes terminal: 2

Nonterminals (types): $\mathbb{R}$ | Terminals: x, 2, +, ln

The symbol | means OR (multiple production rules for one type)

Figure: BNF grammar: rules, nonterminals, and terminals

English Grammar Example

$$\mathbb{S} \mapsto \mathbb{N}\,\mathbb{V}$$
$$\mathbb{V} \mapsto \mathbb{V}\,\mathbb{A}$$
$$\mathbb{A} \mapsto \text{rapidly} \mid \text{efficiently}$$
$$\mathbb{N} \mapsto \text{Alice} \mid \text{Bob} \mid \text{Mykel}$$
$$\mathbb{V} \mapsto \text{runs} \mid \text{reads} \mid \text{writes}$$

Derivation: $\mathbb{S} \to \mathbb{N}\,\mathbb{V} \to \text{Mykel}\,\mathbb{V}\,\mathbb{A} \to$ *"Mykel writes rapidly"*

Grammar Depth and Size

Constraint: Maximum Depth

Expression optimization often constrains tree depth $d_{\max}$ to avoid infinite recursion or excessively large trees.

- **Terminal node**: no children; a leaf of the tree
- **Nonterminal node**: an unexpanded type (blue nodes in figure)
- The number of expressions grows *super-exponentially* with height h
- For $\mathbb{N} \mapsto \{\mathbb{N}, \mathbb{N}\} \mid \{\mathbb{N}, \} \mid \{, \mathbb{N}\} \mid \{\}$:

$$T(h) = 1 + 4 \cdot T(h-1)^2 \qquad \text{(super-exponential)}$$

Four-Function Calculator (Example 23.2)

Grammar (digits 1–9):

$$\mathbb{R} \mapsto \{1, \ldots, 9\}$$
$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$
$$\mathbb{R} \mapsto \mathbb{R} - \mathbb{R}$$
$$\mathbb{R} \mapsto \mathbb{R} \times \mathbb{R}$$
$$\mathbb{R} \mapsto \mathbb{R} / \mathbb{R}$$

An **infinite** number of expressions can be generated (no terminal constraint).

Grammar Implementation in Python

```python
# Represent a grammar as a dict: type_name -> list of production rules
# Each rule is a tuple of (symbol, is_terminal) pairs

grammar = {
    "R": [
        # R -> R + R
        [("R","nt"), ("+","t"), ("R","nt")],
        # R -> x
        [("x","t")],
        # R -> ln(R)
        [("ln","t"), ("R","nt")],
        # R -> 2
        [("2","t")],
    ]
}

def is_terminal(symbol, grammar):
    return symbol not in grammar

def get_rules(symbol, grammar):
    return grammar.get(symbol, [])

# Random expression tree generation (depth-limited)
import random

def rand_tree(symbol, grammar, max_depth=5, depth=0):
    """Recursively build a random expression tree."""
```

Genetic Programming — Overview

Key Idea

Apply genetic algorithms to **expression trees**. Populations of trees evolve via selection, crossover, and mutation.

Algorithm Skeleton:

1. Initialize population of random trees (grammar-constrained)
2. Evaluate objective $f(\text{tree})$ for each individual
3. Select parents (truncation, tournament, etc.)
4. Apply **tree crossover** to produce offspring
5. Apply **tree mutation** and/or **tree permutation**
6. Repeat until convergence

Key Difference from GA

Individuals are **variable-length trees**, not fixed-length vectors. Crossover and mutation must preserve type-correctness enforced by the grammar.

Reference: Koza (1992); also used in symbolic regression (e.g. PySR, gplearn).

Tree Crossover

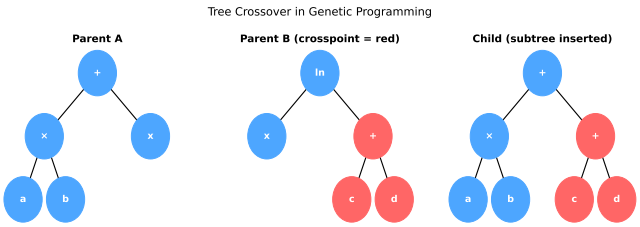

Figure: Tree crossover: a subtree from Parent B is inserted into Parent A at a type-compatible location.

Algorithm 23.1 (Tree Crossover)

1. Deep-copy parent A as child
2. Sample a random node (crosspoint) from B
3. Get type of crosspoint in B
4. Find compatible insertion location in child
5. Insert deep-copy of crosspoint subtree

Constraint: Replacement nodes must have the **same return type**; only nodes within $d_{\text{max}} - d_{\text{subtree}}$ depth are valid locations.

Tree Mutation

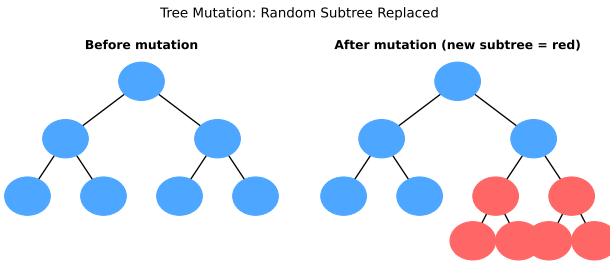

Figure: Tree mutation: a random subtree is deleted and replaced by a newly generated subtree of the same type.

Algorithm 23.2 (Tree Mutation)

Given mutation probability p:

1. Deep-copy individual a
2. With prob. p: sample random node location
3. Get type at that location
4. Generate new random subtree of same type
5. Insert new subtree at location

Tree crossover tends to produce deeper trees; mutation introduces **new genetic material**.

Tree Permutation

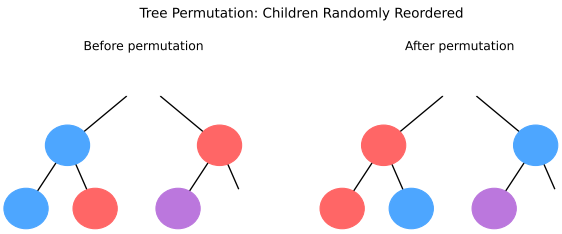

Figure: Tree permutation: children of a randomly chosen node are randomly reordered.

Algorithm 23.3 (Tree Permutation)

Given mutation probability p:

1. Deep-copy individual a
2. With prob. p: sample random node
3. Count children n; get child types
4. Use Fisher-Yates-style swap among type-compatible children
5. Return permuted child

Note: Only children with **matching types** may be swapped. Often combined with tree mutation.

Genetic Programming: Python Implementation

```python
import random, copy, math
import numpy as np

# ---- Tree node representation ----
class Node:
    def __init__(self, rule, children=None):
        self.rule = rule          # production rule applied
        self.children = children or []

# ---- Tree crossover ----
def tree_crossover(parent_a, parent_b, grammar, max_depth=10):
    child = copy.deepcopy(parent_a)
    # Sample crosspoint from parent_b
    crosspoint = random_node(parent_b)
    typ = return_type(crosspoint, grammar)
    d_subtree = tree_depth(crosspoint)
    d_max = max_depth + 1 - d_subtree
    if d_max > 0:
        loc = find_location(child, grammar, typ, d_max)
        if loc is not None:
            insert(child, loc, copy.deepcopy(crosspoint))
    return child

# ---- Tree mutation ----
def tree_mutate(individual, grammar, p_mutate=0.1, max_depth=10):
    child = copy.deepcopy(individual)
    if random.random() < p_mutate:
```

Example 23.5: Approximating π with Genetic Programming

```python
import math, random

# Grammar: digits 1-9 + arithmetic ops
grammar_pi = {
    "R": [
        [("R","nt"),("+","t"),("R","nt")],
        [("R","nt"),("-","t"),("R","nt")],
        [("R","nt"),("*","t"),("R","nt")],
        [("R","nt"),("/","t"),("R","nt")],
        # Terminals: digits 1-9
        *[[(str(d),"t")] for d in range(1,10)],
    ]
}

def objective(tree):
    val = eval_expr(tree)
    if not math.isfinite(val):
        return float('inf')
    delta = abs(val - math.pi)
    return math.log(delta + 1e-15) + len(tree)/1000

# Run genetic programming
pop_size, max_iter = 1000, 200
population = [rand_tree("R", grammar_pi, max_depth
    =10)
              for _ in range(pop_size)]
for gen in range(max_iter):
```

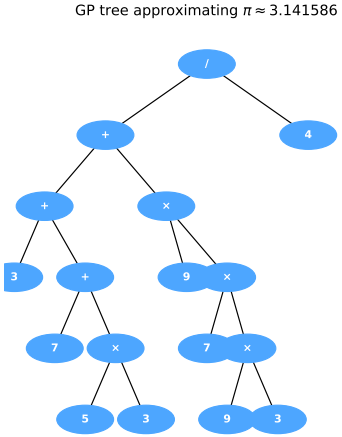

Figure: Best GP tree: evaluates to ≈ 3.141586

Grammatical Evolution — Key Idea

Integer Array Instead of Tree

Grammatical evolution operates on an **integer array** (chromosome) rather than directly on expression trees.

Decoding Process:

1. Start at root type
2. At each nonterminal, use the next integer in the array to **select a production rule**

$$\text{rule index} = x[c] \bmod {}_1|\text{rules}|$$

(1-indexed modular arithmetic)

3. Increment counter c; recurse on children
4. Stop when all nonterminals are expanded or $c > c_{\max}$

Advantages over GP

- Standard GA operators (crossover, mutation) apply directly to integer arrays
- Grammar guarantees syntactic validity
- Integer representation is simpler to implement

Key Parameters

x = integer design vector
$c_{\max}$ = max rule applications (prevents infinite loops)

Grammatical Evolution: Decoding (Algorithm 23.4)

Algorithm: decode(x, $\mathcal{G}$, sym, $c_{\max}$)

1. types $\leftarrow \mathcal{G}$[sym] (rules for type sym)
2. If $|\text{types}| > 1$: $g \leftarrow x[c \bmod {}_1|x|]$; $c \mathrel{+}= 1$
3. rule $\leftarrow$ types[$g \bmod {}_1|\text{types}|$]
4. node $\leftarrow$ RuleNode(rule)
5. For each child type in rule (if $c < c_{\max}$):
   recurse: cnode $\leftarrow$ decode($x, \mathcal{G}, \text{ctyp}, c_{\max}, c$)
6. Return DecodedExpression(node, c)

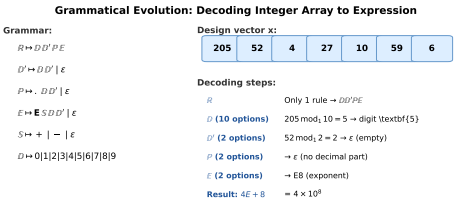

Figure: Decoding $[205, 52, 4, 27, 10, 59, 6]$ to expression $4E{+}8$

Grammatical Evolution: Worked Example

Example 23.6: Real-Valued String Grammar

Grammar for real-valued numbers:

$$\mathbb{R} \mapsto \mathbb{D}\,\mathbb{D}'\,\mathbb{P}\,\mathbb{E}, \quad \mathbb{D}' \mapsto \mathbb{D}\,\mathbb{D}' \mid \epsilon, \quad \mathbb{P} \mapsto .\,\mathbb{D}\,\mathbb{D}' \mid \epsilon$$

$$\mathbb{E} \mapsto \mathbf{E}\,\mathbb{S}\,\mathbb{D}\,\mathbb{D}' \mid \epsilon, \quad \mathbb{S} \mapsto + \mid - \mid \epsilon, \quad \mathbb{D} \mapsto 0|1|2|3|4|5|6|7|8|9$$

Design vector: $[205, 52, 4, 27, 10, 59, 6]$

Step	Choice	Result
$\mathbb{R}$	1 rule only	$\mathbb{D}\mathbb{D}'\mathbb{P}\mathbb{E}$
$\mathbb{D}$	$205 \bmod {}_1 10 = 5$	digit **5**
$\mathbb{D}'$	$52 \bmod {}_1 2 = 2$	ϵ
$\mathbb{P}$	choose ϵ	(no decimal)
$\mathbb{E}$	choose rule	$E + 8$

Result: String $= 4E{+}8 = 4 \times 10^8$

Depth-First Decoding

The implementation is depth-first. If 52 were instead 51: $\mathbb{D}' \mapsto \mathbb{D}\,\mathbb{D}'$, eventually producing $43950.950E{+}8$.

Grammatical Evolution: Python Implementation

```python
import numpy as np

def decode(x, grammar, sym, c_max=1000, c=0):
    """
    Decode integer array x into an expression tree.
    x       : list/array of non-negative integers (chromosome)
    grammar : dict  type -> list of rules
    sym     : starting nonterminal symbol (str)
    c_max   : max rule applications (prevents infinite recursion)
    c       : current position counter (1-indexed modular)
    Returns : (node_dict, c)
    """
    rules = grammar[sym]
    if len(rules) > 1:
        g = x[(c % len(x))]    # 0-indexed Python
        c += 1
        rule = rules[g % len(rules)]
    else:
        rule = rules[0]

    node = {"sym": sym, "rule": rule, "children": []}
    child_types = [s for s, kind in rule if kind == "nt"]

    if child_types and c < c_max:
        for ctyp in child_types:
            cnode, c = decode(x, grammar, ctyp, c_max, c)
            node["children"].append(cnode)
```

GE Operators and Praxis

Standard GA operators on the integer array:

- **Crossover**: single-point or uniform on the integer chromosome
- **Mutation**: flip or add Gaussian noise to each integer
- **Duplication**: append a prefix of the chromosome to itself (inspired by gene duplication in DNA)

Drawbacks of Grammatical Evolution

1. Hard to determine whether a chromosome encodes a valid/complete expression
2. A small change in the chromosome can produce a *large* change in the expression (locality problem)

Algorithm 23.5 (GE Crossover)

Single-point crossover on two integer arrays of type `GrammaticalEvolution`:

1. Sample crossover point
2. Exchange tails
3. Return two new chromosomes

Algorithm 23.6 (GE Mutation)

For each element in the array, add random noise with probability p.

Probabilistic Grammars — Motivation

Idea: Learn Rule Selection Probabilities

Assign a **weight** $w_i^{\mathbb{T}}$ to each production rule i of type $\mathbb{T}$. The sampling probability is:

$$P(\mathbb{T} \mapsto \text{rule}_i) = \frac{w_i^{\mathbb{T}}}{\sum_j w_j^{\mathbb{T}}}$$

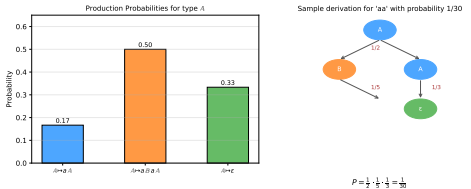

Figure: Rule weights and sampling probabilities for a probabilistic grammar

Optimization approach:

1. Sample a population of expressions
2. Evaluate and select elite samples
3. **Update** the grammar weights based on elite rules used
4. Iterate until convergence (similar to CEM / EDA)

Probabilistic Grammar: Likelihood and Update

Example 23.7: Computing Expression Probability

Grammar for strings of "a"s:

$$\mathbb{A} \mapsto a\,\mathbb{A}\ (w=1), \quad \mathbb{A} \mapsto a\,\mathbb{B}\,a\,\mathbb{A}\ (w=3), \quad \mathbb{A} \mapsto \epsilon\ (w=2)$$

$$\mathbb{B} \mapsto a\,\mathbb{B}\ (w=4), \quad \mathbb{B} \mapsto \epsilon\ (w=1)$$

Sampling sequence for "aa":

$$P(\mathbb{A} \mapsto a\,\mathbb{B}\,a\,\mathbb{A}) \cdot P(\mathbb{B} \mapsto \epsilon) \cdot P(\mathbb{A} \mapsto \epsilon) = \frac{1}{2} \cdot \frac{1}{5} \cdot \frac{1}{3} = \frac{1}{30}$$

Algorithm 23.10: Learning Update

Given an elite set of expressions X_s:

1. Reset all rule weights to 0
2. For each expression $x \in X_s$: increment w_i for every production rule i used in x
3. Result: weights reflect frequency of use in elite expressions

Probabilistic Grammar: Python Code

```python
import numpy as np

class ProbGrammar:
    """Probabilistic context-free grammar with learnable weights."""
    def __init__(self, grammar):
        """
        grammar: dict  type -> list of rules (each rule is a list of (sym, kind))
        """
        self.grammar = grammar
        # Initialize weights uniformly
        self.weights = {typ: np.ones(len(rules))
                        for typ, rules in grammar.items()}

    def sample_rule(self, typ):
        """Sample a production rule index for type `typ`."""
        w = self.weights[typ]
        probs = w / w.sum()
        return np.random.choice(len(self.grammar[typ]), p=probs)

    def sample_expression(self, sym, max_depth=10, depth=0):
        """Recursively sample an expression tree."""
        rules = self.grammar[sym]
        if depth >= max_depth:
            # Force terminal rule
            term_idx = [i for i,r in enumerate(rules)
                        if all(k=="t" for _,k in r)]
            idx = np.random.choice(term_idx) if term_idx else 0
```

Probabilistic Prototype Trees (PPT)

Key Idea

A **probabilistic prototype tree** (PPT) stores a probability vector at *every node* of the expression tree — one probability per applicable production rule at that position.

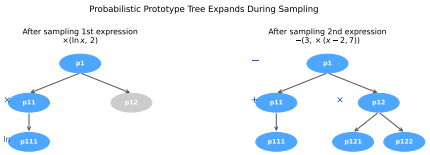

Figure: PPT expands as new expression positions are sampled

Unlike probabilistic grammars (which use *global* rule probabilities), PPTs learn **position-specific** probabilities:

- Each node $\mathbf{p}_{ij\ldots}$ has its own probability vector
- The tree grows on demand when new positions are visited
- Probabilities are initialized uniformly; $w_{\text{terminal}} = 0.6$, $w_{\text{nonterm}} = 0.4$

PPT: Sampling and Target Probability

Target Probability (Algorithm 23.13)

Compute a target probability for the best-known expression:

$$q_{\text{best}} = \left(\frac{1}{n_{\text{rules}}}\right)^{1/m}$$

where m is the number of components in $\mathbf{p}$ and n_{rules} is the total number of rules applied to reach the best expression.

Larger $q_{\text{best}} \Rightarrow$ bias toward top-performing solutions.

Learning Update (Algorithm 23.14)

Iterative EDA-style:

1. Sample m expressions from PPT
2. Evaluate objective
3. Find best expression x_{best}
4. Update PPT toward x_{best}
5. Prune irrelevant branches
6. Repeat

Equation (23.4)

$$f_{\text{target}}(x_s) = \prod_k q_{\text{best},k}^{n_k}$$

PPT: Mutation Update Rule

Mutation Formula (Eq. 23.5)

For each component p_i in a node's probability vector:

$$p_i \; += \; \beta\,(1 - p_i)$$

with probability $p_{\text{mutation}}/(\sqrt{|x_{\text{best}}|} \cdot \sqrt{m})$. Then **renormalize** the vector so it sums to 1.

Interpretation: Smaller probabilities receive *larger* absolute increases (drives toward uniformity for non-selected entries), while selected entries are boosted.

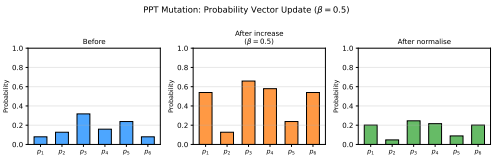

Figure: Before / after increase / after normalize for $\beta = 0.5$

PPT: Pruning (Algorithm 23.16)

Pruning Condition

A PPT subtree can be **pruned** when the probability of its parent choosing a *terminal* rule exceeds a threshold $p_{\text{threshold}}$ (typically 0.99):

$$\max_i p_i^{(\text{parent})} > p_{\text{threshold}}$$

If the dominant rule is terminal, children are unnecessary. If nonterminal, children with mismatched return type are pruned.

Example 23.8: When to Prune

Node with rules:

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$
$$\mathbb{R} \mapsto \ln(\mathbb{R})$$
$$\mathbb{R} \mapsto 2 \mid x$$
$$\mathbb{R} \mapsto \mathbb{S}$$

If $P(2)$ or $P(x)$ becomes large: children of that node become unnecessary and can be pruned.

PPT: Python Implementation

```python
import numpy as np

class PPTNode:
    """Node in a Probabilistic Prototype Tree."""
    def __init__(self, grammar, w_terminal=0.6):
        self.grammar = grammar
        self.ps = {}  # type -> probability vector over rules
        self.children = []
        w_nonterm = 1.0 - w_terminal
        for typ, rules in grammar.items():
            ps_typ = np.array([w_terminal if is_terminal(grammar, i) else w_nonterm
                               for i, _ in enumerate(rules)], dtype=float)
            self.ps[typ] = ps_typ / ps_typ.sum()

    def get_child(self, i):
        """Lazily expand children as needed."""
        while len(self.children) <= i:
            self.children.append(PPTNode(self.grammar))
        return self.children[i]

def sample_from_ppt(ppt, grammar, sym):
    """Sample an expression tree from a PPT node."""
    rules = grammar[sym]
    rule_idx = np.random.choice(len(rules), p=ppt.ps[sym])
    rule = rules[rule_idx]
    child_syms = [s for s, k in rule if k == "nt"]
    node = {"sym": sym, "rule_idx": rule_idx, "children": []}
```

Comparison: GP vs. GE vs. Prob. Grammar vs. PPT

	GP	**GE**	**Prob. Grammar**	**PPT**
Representation	Tree	Integer array	Tree (+ weights)	Tree (+ per-node weights)
Grammar enforced	Yes	Yes	Yes	Yes
Learns structure	No	No	Global weights	Position-specific
Crossover	Tree	Integer	N/A (EDA)	N/A (EDA)
Mutation	Subtree	Integer	N/A	Weight update
Memory overhead	Low	Low	Low	High (tree grows)
Locality	Medium	Poor	Good	Excellent

Key Insight

PPTs trade **memory** for **locality**: every position in the expression tree has its own distribution, enabling fine-grained probability learning.

Chapter 23 — Summary

- **Expression optimization** allows optimizing tree structures that, under a grammar, can represent sophisticated programs and designs lacking a fixed size.
- **Grammars** (BNF / context-free) define the production rules used to construct valid expressions; they guarantee syntactic correctness automatically.
- **Genetic programming** adapts genetic algorithms to perform mutation and crossover on expression trees (tree crossover, tree mutation, tree permutation).
- **Grammatical evolution** operates on an integer array that is decoded into an expression tree, enabling standard GA operators.
- **Probabilistic grammars** learn *global* rule selection probabilities from elite expression samples.
- **Probabilistic prototype trees** learn *position-specific* probabilities for every node in the expression tree via a mutation update rule.

Selected Exercises I

Exercise 23.1

How many expression trees can be generated using the following grammar and starting set $\{\mathbb{R}, \mathbb{I}, \mathbb{F}\}$?

$$\mathbb{R} \mapsto \mathbb{I} \mid \mathbb{F}, \quad \mathbb{I} \mapsto 1 \mid 2, \quad \mathbb{F} \mapsto \pi$$

Solution: Six trees: $\mathbb{I} \to 1$, $\mathbb{I} \to 2$, $\mathbb{F} \to \pi$, $\mathbb{R} \to \mathbb{I} \to 1$, $\mathbb{R} \to \mathbb{I} \to 2$, $\mathbb{R} \to \mathbb{F} \to \pi$.

Exercise 23.2

The number of expression trees up to height h grows super-exponentially. For $\mathbb{N} \mapsto \{\mathbb{N}, \mathbb{N}\} \mid \{\mathbb{N}, \} \mid \{, \mathbb{N}\} \mid \{\}$ the count satisfies $T(h) = 1 + 4\,T(h-1)^2$, giving $T(1) = 5$, $T(2) = 101$, and so on.

Selected Exercises II

Exercise 23.9: Probabilistic Grammar Likelihood

Grammar with weights:

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R} \mid \mathbb{R} \times \mathbb{R} \mid \mathbb{F} \mid \mathbb{I} \qquad w_{\mathbb{R}} = [1, 1, 5, 5]$$
$$\mathbb{F} \mapsto 1.5 \mid \infty \qquad w_{\mathbb{F}} = [4, 3]$$
$$\mathbb{I} \mapsto 1 \mid 2 \mid 3 \qquad w_{\mathbb{I}} = [1, 1, 1]$$

Generation probability of $1.5 + 2$:

$$P(\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}) = 1/12, \qquad P(\mathbb{R} \mapsto \mathbb{F}) = 5/12$$
$$P(\mathbb{F} \mapsto 1.5) = 4/7, \qquad P(\mathbb{R} \mapsto \mathbb{I}) = 5/12$$
$$P(\mathbb{I} \mapsto 2) = 1/3$$

$$P = \frac{1}{12} \cdot \frac{5}{12} \cdot \frac{4}{7} \cdot \frac{5}{12} \cdot \frac{1}{3} = \frac{25}{9072} \approx 0.00276$$

Selected Exercises III

Exercise 23.8: Clock Gear Optimization

Design a gear train to produce second, minute, and hour hands. Objective (penalizes imprecision and tree complexity):

$$\left(\min_{\text{hands}}(1 - t_{\text{hand}})^2\right) + \left(\min_{\text{hands}}(60 - t_{\text{hand}})^2\right) + \left(\min_{\text{hands}}(3600 - t_{\text{hand}})^2\right) + \#\text{nodes} \cdot 10^{-3}$$

Grammar (G_r = gear of radius r, $\mathbb{R}$ = rim, $\mathbb{A}$ = axle, $\mathbb{H}$ = hand):

$$G_r \mapsto \mathbb{R}\,\mathbb{A} \mid \epsilon, \quad \mathbb{R} \mapsto \mathbb{R}\,\mathbb{R} \mid G_r \mid \epsilon, \quad \mathbb{A} \mapsto \mathbb{A}\,\mathbb{A} \mid G_r \mid \mathbb{H} \mid \epsilon$$

A minimal solution uses gears G_{25}, G_{10}, G_{100} to produce rotation periods of 1s, 60s, and 3600s.

Exercise 23.8: Python Approach

```python
import numpy as np, math, random

# Grammar for gear train design
gear_grammar = {
    "G": [  # gear of some radius
        [("R","nt"), ("A","nt")],  # G -> rim + axle
        [],                         # G -> epsilon
    ],
    "R": [  # rim
        [("R","nt"), ("R","nt")],  # R -> R R
        [("G","nt")],               # R -> G
        [],                         # R -> epsilon
    ],
    "A": [  # axle
        [("A","nt"), ("A","nt")],  # A -> A A
        [("G","nt")],               # A -> G
        [("H","t")],                # A -> hand
        [],                         # A -> epsilon
    ],
}

def gear_objective(tree, radii):
    """
    Evaluate gear tree and compute period mismatch.
    radii: dict mapping gear_id -> radius
    """
    periods = get_hand_periods(tree, radii)
```

Chapter 23: Key Takeaways

What you should know:

1. How to write a BNF grammar for a problem domain
2. How GP crossover and mutation maintain type-correctness
3. How GE decodes an integer array via modular indexing
4. How probabilistic grammars update weights from elite samples
5. How PPTs extend probabilistic grammars with per-node distributions

Connections to other chapters:

- Ch. 9 (Genetic Algorithms): GP is GA applied to trees
- Ch. 9 (EDAs / CEM): probabilistic grammars and PPTs are EDAs over expression spaces
- Ch. 7 (Direct Methods): GE converts expression search to a standard real-valued search

Take-Home Message

Grammars are the key: they enforce syntactic validity so that **every evaluated expression is legal**, regardless of which optimization operator is applied.